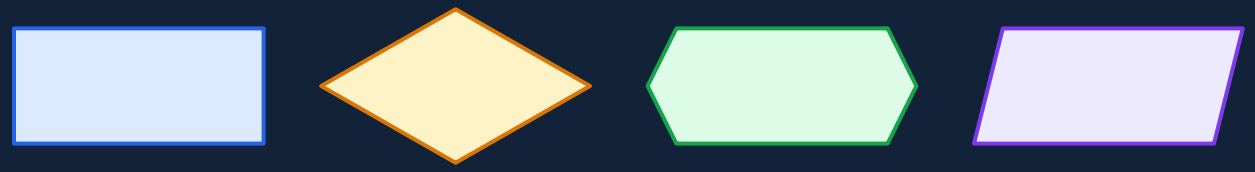
Diagramas de flujo y Nassi-Shneiderman: del código al dibujo

Clase 15 · Jueves 29 de octubre de 2026 · Aula



Notica tech

Microsoft to offer buyouts to 60 000 workers in new cost cut

Some personnel in the US, Canada, India and the UK at a vice president level and below will be eligible, said a Microsoft spokesperson.

Microsoft will offer buyouts to about 60 000 workers, representing roughly a quarter of its total staff, as part of the company's efforts to trim costs and retool its workforce for the artificial intelligence era.

Some personnel in the US, Canada, India and the UK at a vice president level and below will be eligible, said a Microsoft spokesperson.

Objetivo de la sesión

Ver el mismo algoritmo de tres formas y pasar de una a otra sin perder nada.

Python ⇄ **Diagrama de flujo** ⇄ **Nassi-Shneiderman**

1 · Código

Instrucciones en un lenguaje de programación en particular

2 · Dibujar el camino

Diagrama de flujo: figuras unidas por flechas que muestran por dónde va la ejecución.

3 · Encajar bloques

Nassi-Shneiderman: bloques que se meten unos dentro de otros, sin flechas.

Tres formas de escribir **lo mismo**: si cambias una, las otras dos deben cambiar igual.

Antes de empezar: ¿el idioma moldea el pensamiento?

La hipótesis de Sapir-Whorf: Edward Sapir y su alumno Benjamin Lee Whorf (décadas de 1920 a 1940).

Versión fuerte · determinismo lingüístico

El idioma **determina** lo que puedes pensar: si no tienes la palabra, no puedes tener el concepto.

Descartada

Versión débil · relatividad lingüística

El idioma **influye** en lo que notas, recuerdas y en cómo agrupas las cosas, sin impedirte pensar lo demás.

Con evidencia experimental

La pregunta para hoy

Si el idioma influye en cómo pensamos, ¿influye también **el nombre** que le ponemos a una función en cómo pensamos el algoritmo?

Evidencia: el idioma sí influye (versión débil)

Tres estudios a favor y uno que no resistió la réplica.

Azules en ruso

Winawer et al., 2007 · PNAS

El ruso obliga a distinguir *siniy* (azul oscuro) de *goluboy* (azul claro). Los hablantes de ruso distinguen más rápido dos azules que cruzan esa frontera; los de inglés, no.

«Se rompió el florero»

Fausey y Boroditsky, 2008 y 2011

En inglés se usualmente se dice *He broke the vase*; en español, *se rompió el florero*. Vieron videos de personas que rompían cosas a propósito o sin querer. Cuando fue **sin querer**, los hablantes de inglés recordaron mejor **quién** lo hizo; cuando fue a propósito, ambos grupos recordaron igual.

La palabra ayuda a ver

Lupyan y Ward, 2013 · PNAS

Mostraron imágenes con una técnica que las vuelve invisibles (otro ojo ve destellos que las tapan). Oír antes la palabra correcta («calabaza») hizo que la gente **sí notara** la imagen más veces.

⚠️ Experimento fallido

Boroditsky, 2001

El mandarín usa «arriba / abajo» para el tiempo (el mes pasado es «el mes de arriba»). Boroditsky reportó que por eso sus hablantes piensan el tiempo en vertical. Otros equipos repitieron el experimento y no obtuvieron lo mismo (Chen, 2007; January y Kako, 2007).

Lo que sí está respaldado: las palabras guían **qué notamos y qué recordamos**. No que sin palabra no haya idea.

¿Y en programación? El nombre cambia cómo se entiende el código

Experimentos y estudios con programadores profesionales.

Palabras completas

Hofmeister, Siegmund y Holt, 2017 · SANER

72 programadores profesionales buscaron errores en código. Con identificadores de palabras completas fueron **~19% más rápidos** que con nombres de una sola letra (`c`) o abreviaturas (`cnt`).

Nombres que mienten

Arnaoudova, Di Penta y Antoniol, 2016 · EMSE

Catálogo de **17 antipatrones lingüísticos**: el nombre promete una cosa y el código hace otra (p. ej. un `get…` que no regresa nada). De los desarrolladores encuestados, 69% los calificó como mala práctica.

Bugs de nombre

Høst y Østvold, 2009 · ECOOP

Proponen tratar un nombre que no corresponde con lo que hace el código como **un bug más**, que se puede detectar. Ejemplo real: un método llamado `containsField` («¿contiene el campo?») en realidad *buscaba y regresaba* el campo; debió llamarse `findField`.

El verbo es una promesa sobre lo que hace la función: si el código no la cumple, confunde a quien lo lee.

Nombrar con estructura

¿Una plantilla para el nombre ayuda a pensar mejor qué hace la función?

Nombrar es difícil

Feitelson et al., 2022 · IEEE TSE

334 participantes. Ante la misma variable, sólo **6.9%** de las veces dos personas eligieron el mismo nombre. Propusieron 3 pasos: decidir qué conceptos incluir → elegir una palabra para cada uno → armar el nombre. Quienes usaron los pasos hicieron nombres que otros jueces prefirieron **2 a 1**.

El verbo es la norma

AlSuhaibani et al., 2021 · ICSE

Encuesta a más de 1,100 desarrolladores profesionales: **84.6%** está de acuerdo en que el nombre de un método debe contener un verbo que describa su comportamiento (12.7% en desacuerdo).

Lo que dicen los expertos

Kernighan y Pike, 1999: usa nombres activos para funciones, un verbo seguido quizá de un sustantivo.

McConnell, 2004: verbo fuerte + objeto; si cuesta nombrar la rutina, quizá su propósito no está claro.

(Recomendaciones, no experimentos. Paráfrasis.)

Lo más cercano a nuestra regla: **seguir una estructura para nombrar produjo mejores nombres** (Feitelson et al.).

¿Qué tan sólida es la evidencia?

Para decidir cuánto peso darle a la idea.

Afirmación	Tipo de evidencia	Solidez
El idioma determina lo que puedes pensar (Whorf fuerte)	Contraejemplos; consenso en lingüística	✗ Descartada
El idioma influye en percepción y memoria (Whorf débil)	Experimentos controlados; algunos no se replican	◐ Moderada
Los nombres afectan qué tan rápido y bien se entiende el código	Experimentos controlados con programadores	● Sólida
Seguir una estructura al nombrar produce mejores nombres	Un experimento (Feitelson et al., 2022)	◐ Prometedora
«Verbo + complemento» es la convención aceptada	Encuesta y guías de estilo: opinión, no causa	◐ Consenso
Nombrar con verbo + complemento mejora el pensamiento algorítmico de estudiantes	No encontré un estudio que lo pruebe directamente	○ Hipótesis

Conclusión: es una analogía razonable, no un resultado demostrado. La usamos como guía para pensar, no como ley.

A partir de hoy: verbo en infinitivo + complemento

Aplica a todas las funciones del curso. **Única excepción:** las que regresan True / False se nombran `es_` + complemento.

complemento

```
def calcular_promedio_goles(lista_goles):
```

Verbo en infinitivo	Objeto	Contexto	Parámetros
¿qué hace?	¿qué obtiene?	¿de qué?	¿con qué datos?

Por qué

El verbo te obliga a responder **¿qué hace?** antes de pensar **¿cómo lo hace?** Si no encuentras el verbo, todavía no entiendes el problema.

Infinitivo

Termina en **-ar, -er, -ir**: `contar`, `obtener`, `convertir`.

No `cuenta` (conjugado) ni `contando` (gerundio).

Complemento = objeto + contexto

Objeto (obligatorio): lo que obtiene, `promedio`.

Contexto (si hace falta): ¿de qué?, `goles`.

En `contar_victorias` el contexto sobra.

Antes → después: los ejemplos de la clase pasada

Mismo código, nombre nuevo. El verbo dice qué hace la función.

Antes	Después	Comentario
~~primera_cancion~~	**obtener_primera_cancion**	Era sólo un sustantivo
es_mayor_edad	**es_mayor_edad**	✓ Excepción: regresa True / False
~~plan_de_viernes~~	**elegir_plan_viernes**	if / elif / else: elige un camino
~~acceso_concierto~~	**revisar_acceso_concierto**	ifs anidados: revisa paso a paso
~~duracion_playlist~~	**sumar_duracion_playlist**	Acumulador de suma
~~mejor_vuelta~~	**encontrar_mejor_vuelta**	Acumulador del mínimo
~~promedio_goles~~	**calcular_promedio_goles**	Varios acumuladores + fórmula
contar_victorias	**contar_victorias**	✓ Ya cumplía la regla (sin contexto)

verbo + **objeto** + **contexto** (si hace falta)

Única excepción: True / False

Si la función regresa **True** o **False**, se nombra como en la industria: `es_` + **complemento**.

`es_mayor_edad`, `es_par`, `es_valida`

Se lee como una pregunta:

```
if es_par(numero):
```

El verbo y su objeto casi te dicen la estructura

Elegir verbo + objeto es el primer paso del algoritmo.

`obtener_primero`
`obtener_ultimo`

Sacar un dato (validando antes el edge case)

`es_`
`(excepción)`

Pregunta de sí / no → regresa True o False

`asignar_medalla`

Varios caminos → if / elif / else

`contar_victorias`
`contar_reprobados`

Contador: empieza en 0, suma 1 si se cumple algo

`sumar_total`
`sumar_duracion`

Acumulador de suma: total = total + x

`encontrar_minimo`
`encontrar_maximo`

Mínimo o máximo: empieza con el primer elemento

`calcular_promedio`

Dos acumuladores (total y cantidad) + división

`filtrar_aprobados`

Lista nueva: empieza en [] y hace append

⚠️ **Evita**

`hacer_cosas`, `procesar_datos`: verbo vacío

`calculando_total`: gerundio

`calcula_total`: conjugado

`total`: sin verbo

`funcion_1`: no dice nada

«**Contar** victorias» → contador. «**Encontrar** la mejor vuelta» → acumulador del mínimo. El nombre ya es medio algoritmo.

Del problema al nombre (1)

Lee el problema y piensa el nombre antes de ver la respuesta: **verbo + objeto + contexto**.

Problema 1

Recibe la lista de precios de un carrito de compras y regresa **cuánto hay que pagar** en total.



Estructura: acumulador de suma

Problema 2

Recibe las calificaciones de un grupo y regresa **cuántos alumnos reprobaron** (menos de 6).



Estructura: contador con un `if`

Problema 3

Recibe los puntajes de un videojuego y regresa **el más alto** (el récord).



Estructura: acumulador del máximo

Primero **verbo + objeto**: ya dicen la estructura. El contexto dice sobre qué datos.

Del problema al nombre (2)

El contexto se agrega sólo si hace falta. Y si regresa True / False, se usa **es_**.

Problema 4

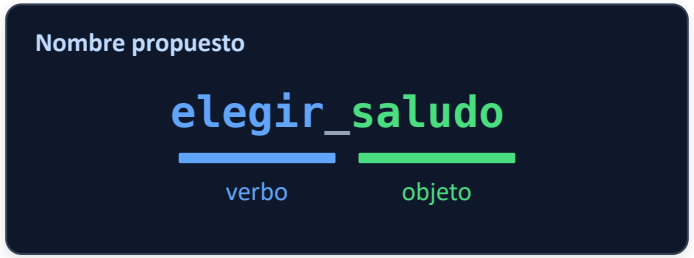
Recibe los mensajes de un chat y regresa **el último**. Si el chat está vacío, regresa `None`.



Estructura: edge case + acceso por índice

Problema 5

Recibe la hora del día (0 a 23) y regresa `"buenos días"`, `"buenas tardes"` o `"buenas noches"`.



Estructura: `if / elif / else`

Problema 6

Recibe un año y regresa `True` si es bisiesto y `False` si no.



Estructura: `if` anidados; regresa True / False

En `elegir_saludo` no hace falta contexto: el objeto ya lo dice todo. Igual que en `contar_victorias`.

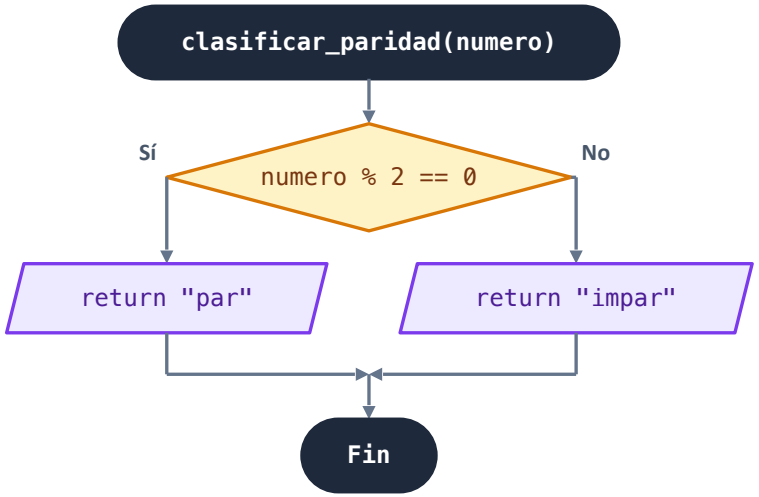
Dos formas de dibujar un algoritmo

El mismo `clasificar_paridad` dibujado de las dos maneras.

Diagrama de flujo

Goldstine y von Neumann, 1947 · norma ISO 5807.

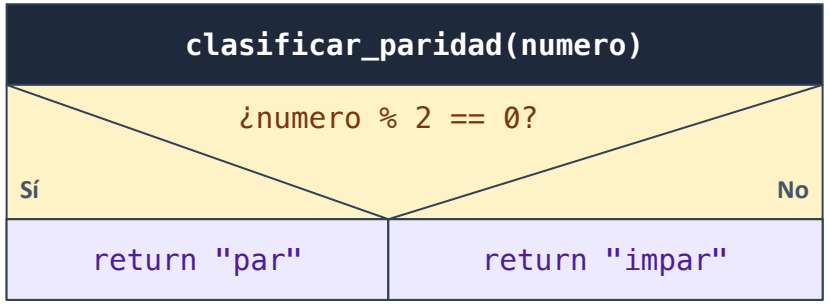
Muestra **el camino**: figuras unidas por flechas. Las flechas pueden ir a cualquier parte, así que también permite dibujar saltos desordenados.



Nassi-Shneiderman

Isaac Nassi y Ben Shneiderman, 1973.

Muestra **la estructura**: bloques que se anidan, sin flechas. Sólo permite secuencia, decisión y ciclo.



Símbolos del diagrama de flujo

Cada figura corresponde a algo que ya sabes escribir en Python.



Terminal

Inicio y fin de la función.

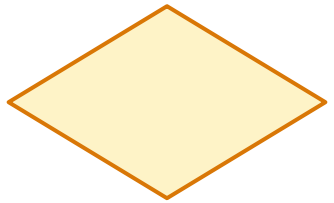
```
def nombre(...):
```



Proceso

Una instrucción: asignar o calcular.

```
total = 0
```



Decisión

Pregunta de sí / no. Siempre dos salidas.

```
if condicion:
```



Ciclo

Repetir para cada elemento de la lista.

```
for x in lista:
```



Salida

El resultado que regresa la función.

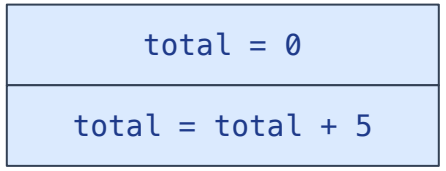
```
return total
```


Flecha: el orden en que se ejecutan las instrucciones. Una flecha que **regresa hacia arriba** es un ciclo.

Bloques de Nassi-Shneiderman

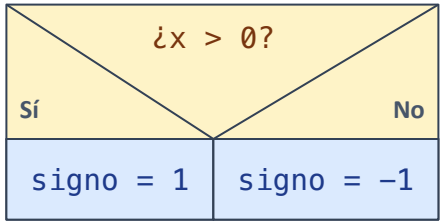
Sin flechas: el orden es de arriba a abajo y lo que está dentro de un bloque le pertenece.

Proceso



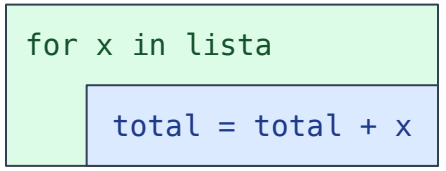
Una instrucción. Varias seguidas = secuencia.

Decisión



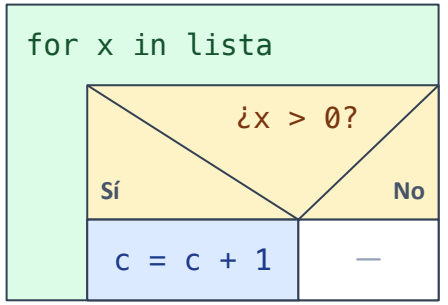
El triángulo hace la pregunta; abajo, una columna por respuesta.

Ciclo



La «L» abraza las instrucciones que se repiten.

Anidado



Un bloque dentro de otro, como la sangría en Python.

Regla de oro: un bloque **nunca se cruza** con otro. O va **debajo** (secuencia) o va **dentro** (anidado).

1 · Secuencia

Cada instrucción es una caja. Se ejecutan de arriba a abajo, una sola vez.

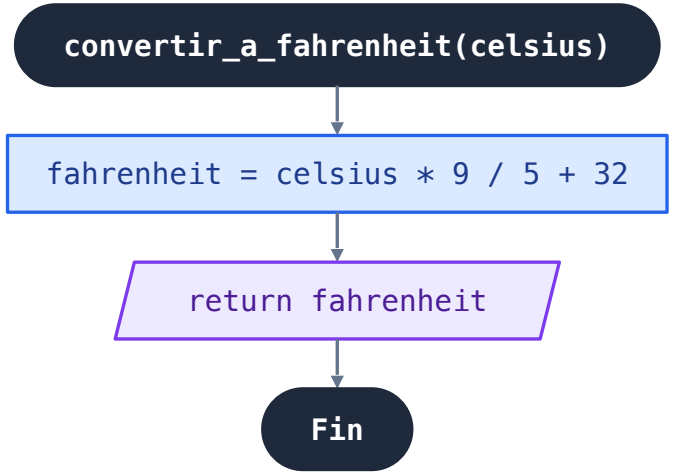
Python · convertir_a_fahrenheit(25) → 77.0

```python
def convertir_a_fahrenheit(celsius):
    fahrenheit = celsius * 9 / 5 + 32
    return fahrenheit
```

Nassi-Shneiderman

convertir_a_fahrenheit(celsius)
fahrenheit = celsius * 9 / 5 + 32
return fahrenheit

Diagrama de flujo



2 · `if` sin `else` · valor por defecto

La rama **No** no tiene caja: salta directo al `return`. En N-S esa columna queda vacía (—).

Python · aplicar_descuento(100, True) → 80.0

```python
def aplicar_descuento(precio, es_estudiante):
    total = precio  # valor por defecto
    if es_estudiante:
        total = precio * 0.8
    return total
```

Nassi-Shneiderman

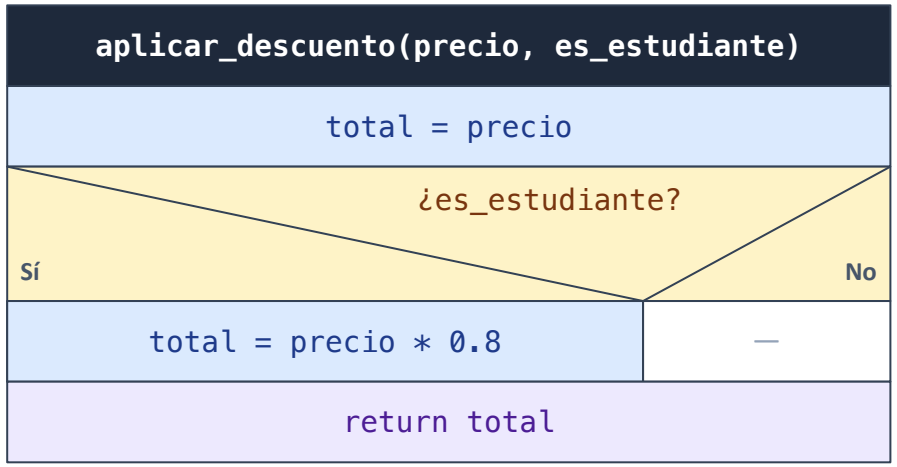
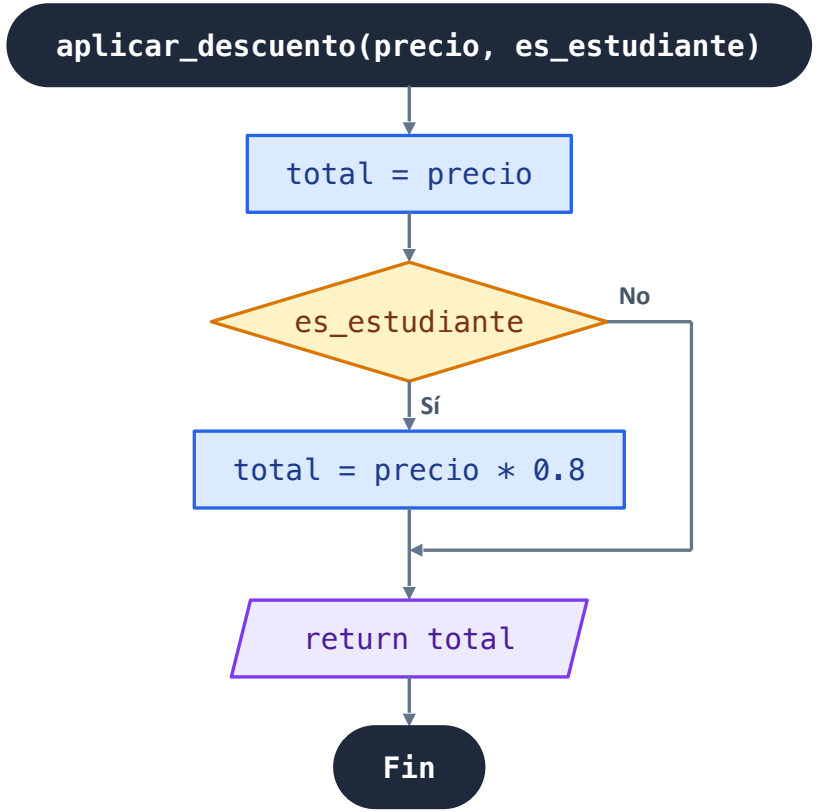


Diagrama de flujo



3 · `if` sin `else` · validar un edge case

El `return` del edge case lleva a su propio **Fin**. En N-S, un `return` termina la función ahí mismo.

Python · obtener_primera_cancion([]) → None

```python
def obtener_primera_cancion(lista_playlist):
    if len(lista_playlist) == 0:  # edge case
        return None  # termina aquí
    return lista_playlist[0]
```

Nassi-Shneiderman

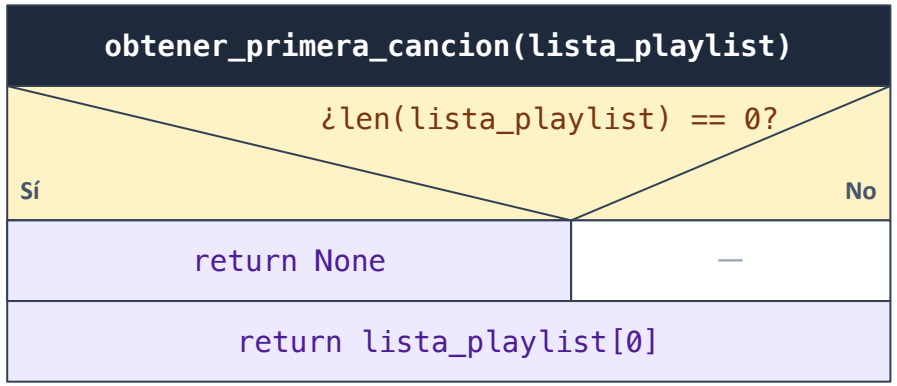
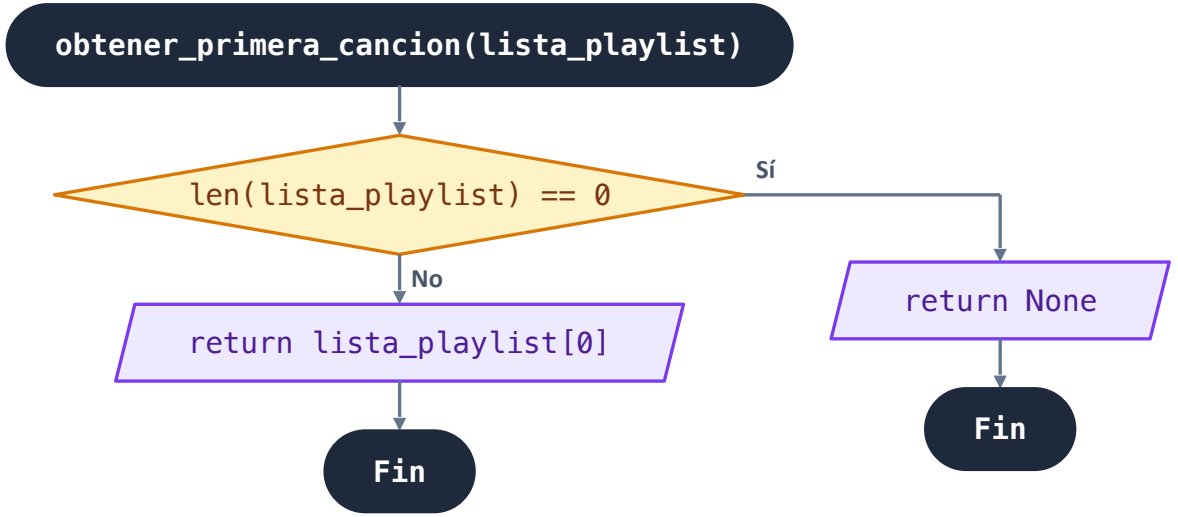


Diagrama de flujo



4 · if / else

Siempre se toma **uno** de los dos caminos. En N-S, cada columna es un camino.

Python · clasificar_paridad(7) → impar

```python
def clasificar_paridad(numero):
    if numero % 2 == 0:
        return "par"
    else:
        return "impar"
```

Nassi-Shneiderman

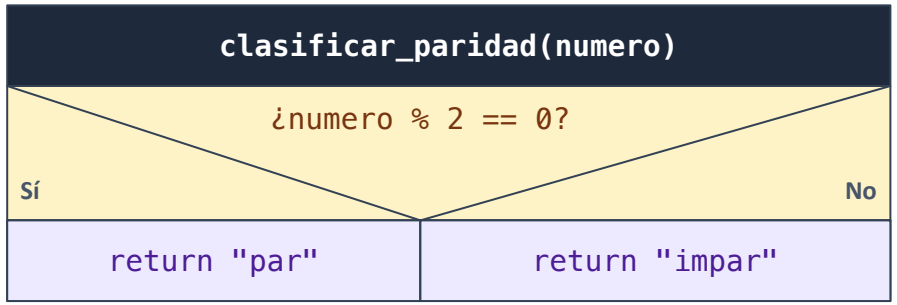
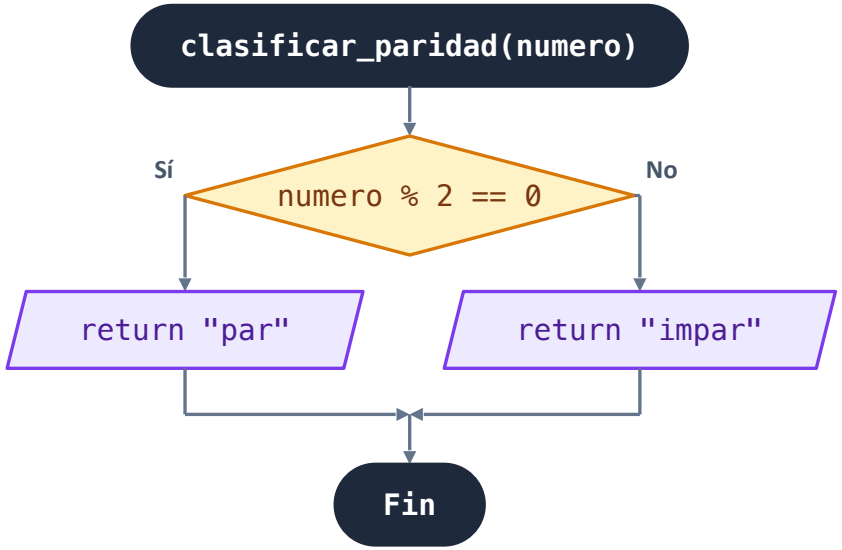


Diagrama de flujo



5 · if / elif / else

Un `elif` es un `if` dentro del `else`: en N-S cada pregunta se anida en la columna **No**.

Python · asignar_medalla(2) → plata

```python
def asignar_medalla(lugar):
    if lugar == 1:
        return "oro"
    elif lugar == 2:
        return "plata"
    elif lugar == 3:
        return "bronce"
    else:
        return "sin medalla"
```

En orden

Python revisa de arriba a abajo y ejecuta **sólo el primer** bloque que se cumple. En los dos diagramas: en cuanto hay un **Sí**, se sale hacia el Fin.

Nassi-Shneiderman

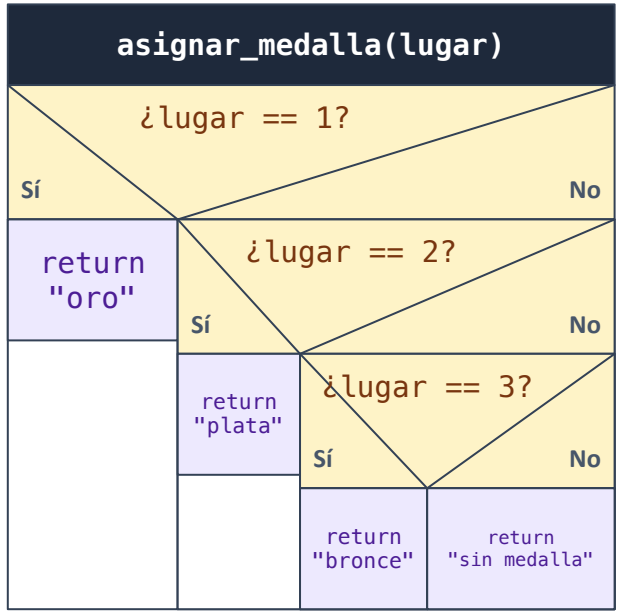
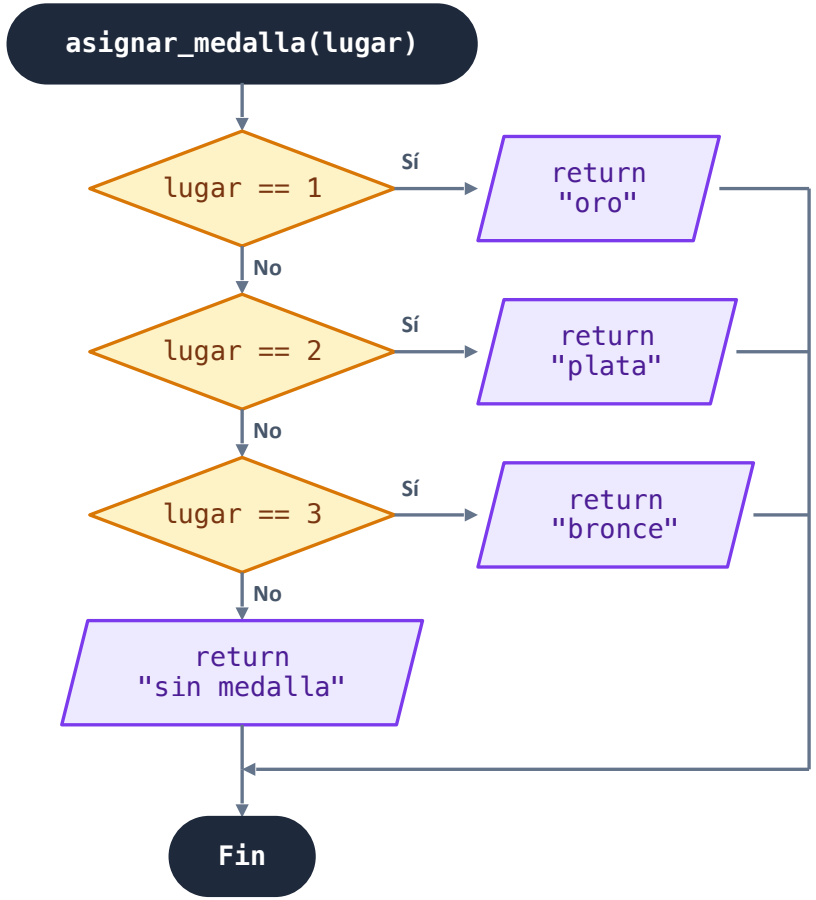


Diagrama de flujo



6 · `if` anidados

Entrada a un concierto. Pulsera roja: mayor de edad. Azul: menor. La edad sólo se revisa **si** tiene boleto.

Python · revisar_acceso(True, 17) → pulsera azul

```python
def revisar_acceso(tiene_boleto, edad):
    if tiene_boleto:
        if edad >= 18:
            return "pulsera roja"
        else:
            return "pulsera azul"
    else:
        return "no pasa"
```

Nassi-Shneiderman

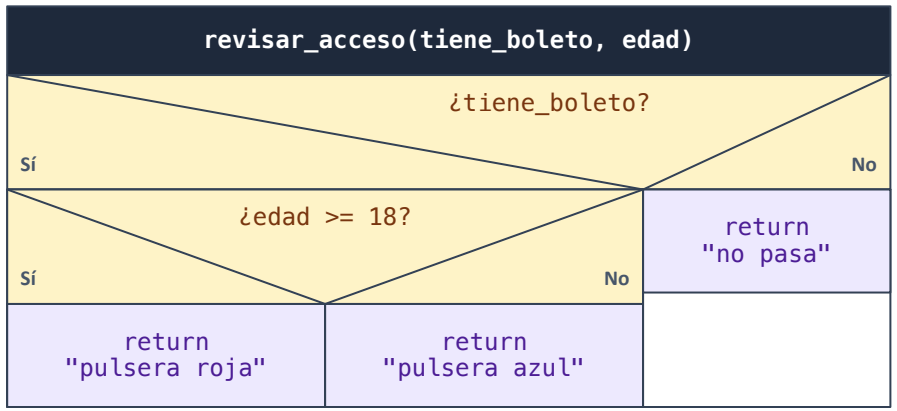
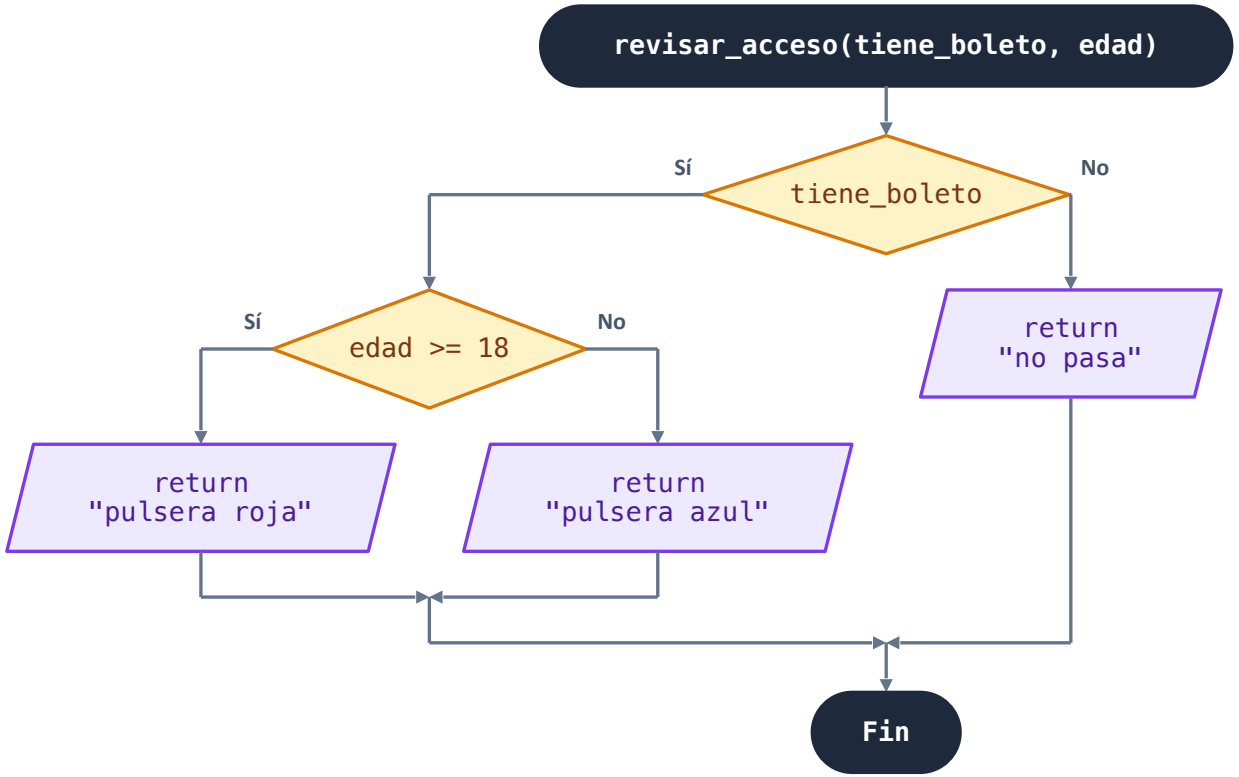


Diagrama de flujo



7 · Acumulador de suma

Los números marcan los 4 pasos: 1 valor inicial · 2 recorrer · 3 actualizar · 4 regresar. Están en el mismo lugar en las tres versiones.

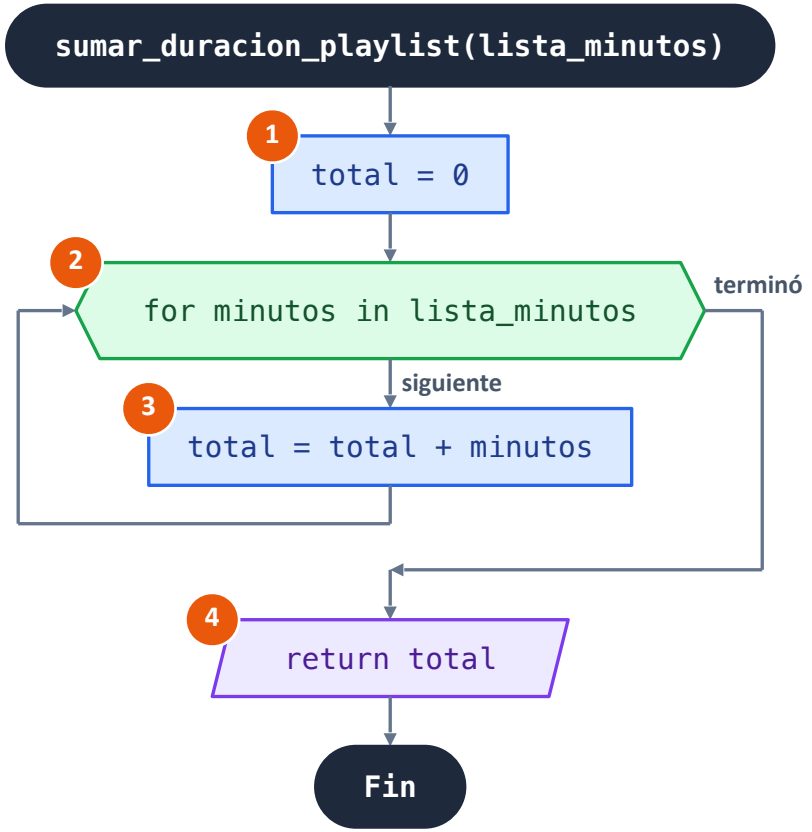
Python · sumar_duracion_playlist([3, 4, 5]) → 12

```python
def sumar_duracion_playlist(lista_minutos):
    total = 0                          # 1
    for minutos in lista_minutos:      # 2
        total = total + minutos        # 3
    return total                       # 4
```

Nassi-Shneiderman



Diagrama de flujo



8 · Contador

Un `if` dentro del ciclo: sólo suma 1 cuando el resultado es "G" (G = ganó, P = perdió, E = empate).

Python · contar_victorias(["G", "P", "G", "E", "G"]) → 3

```python
def contar_victorias(lista_resultados):
    victorias = 0
    for resultado in lista_resultados:
        if resultado == "G":
            victorias = victorias + 1
    return victorias
```

Nassi-Shneiderman

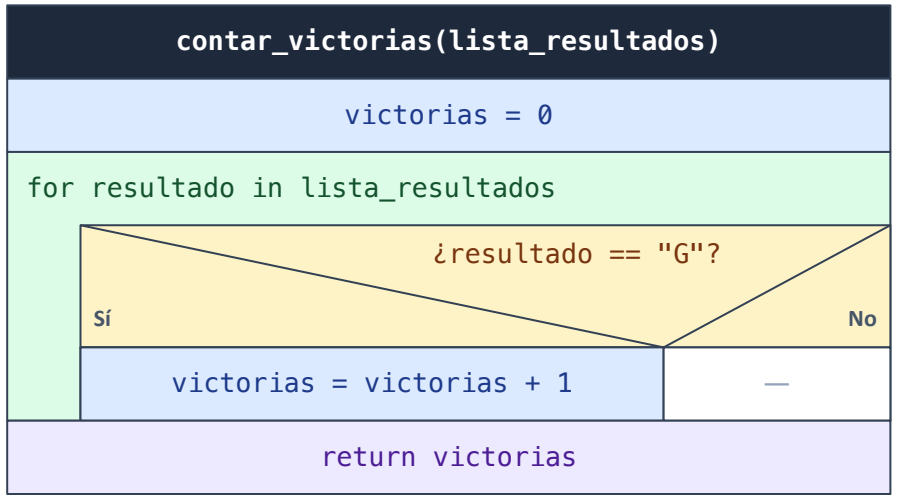
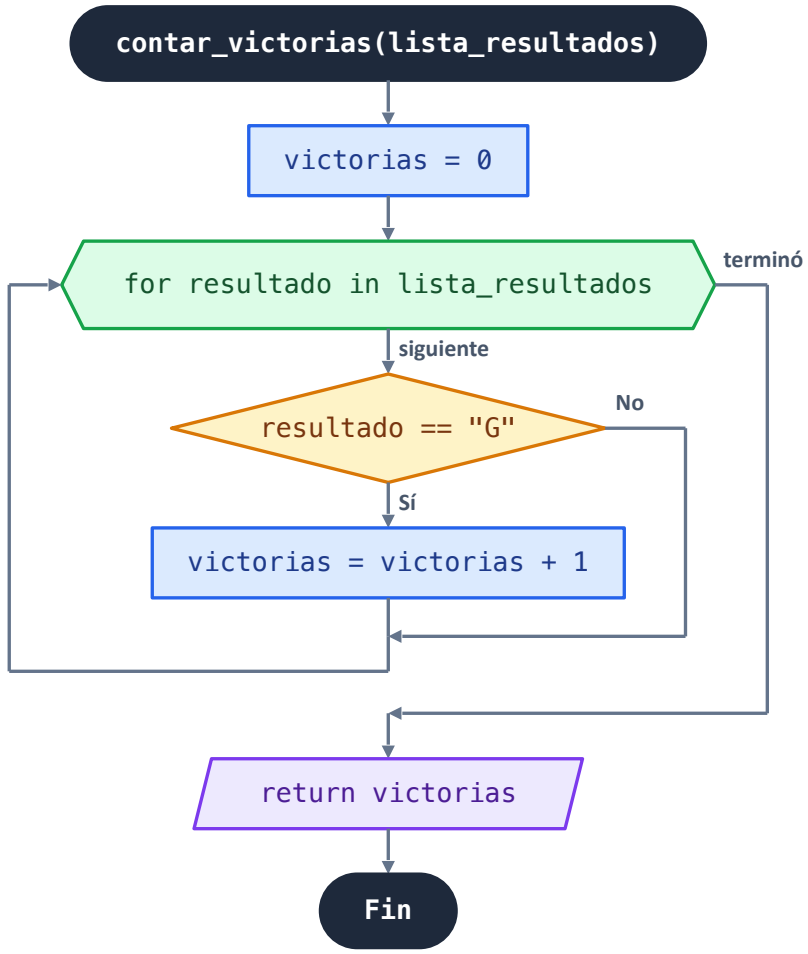


Diagrama de flujo



9 · Mínimo

La mejor vuelta es la de menos segundos. Empieza con el primer elemento y lo reemplaza cada vez que encuentra uno menor.

Python · encontrar_mejor_vuelta([92, 87, 95]) → 87

```python
def encontrar_mejor_vuelta(lista_segundos):
    mejor = lista_segundos[0]
    for segundos in lista_segundos:
        if segundos < mejor:
            mejor = segundos
    return mejor
```

Nassi-Shneiderman

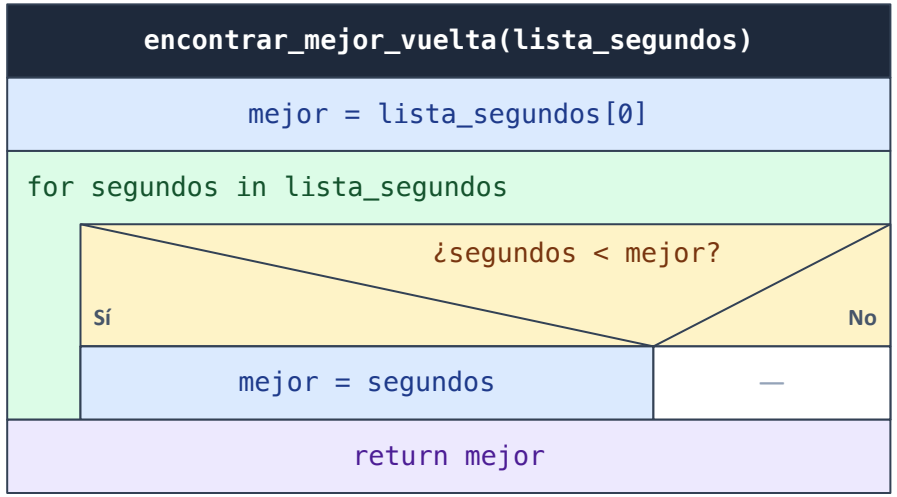
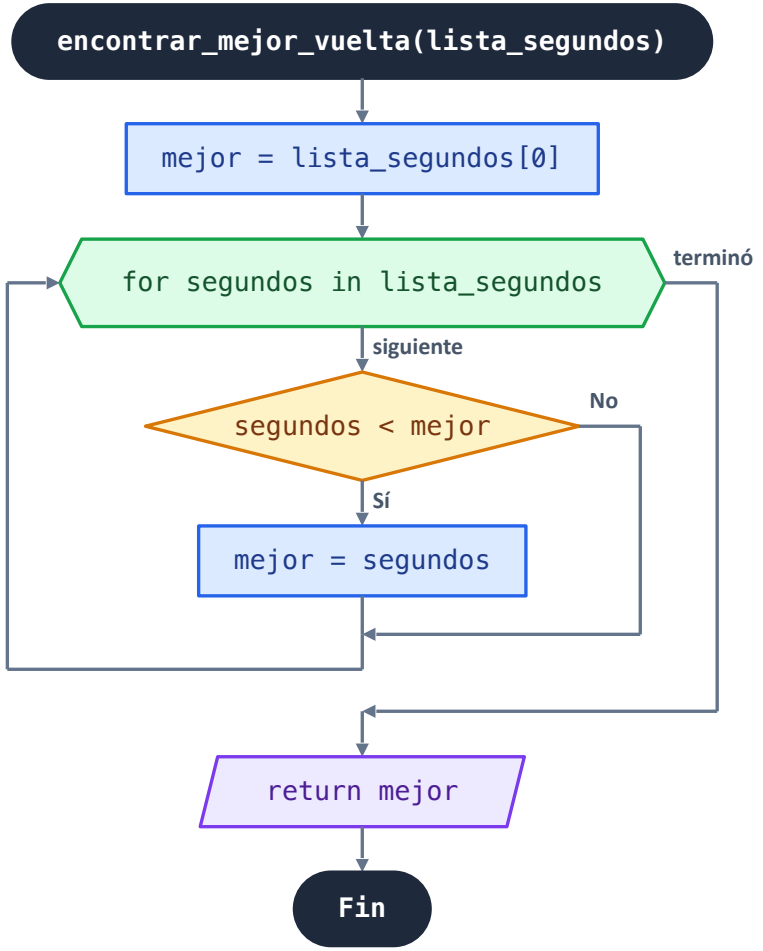


Diagrama de flujo



10 · Máximo con edge case

Combinamos dos estructuras: primero la validación, después el acumulador del máximo.

Python · encontrar_maximo([18, 25, 21]) → 25

```python
def encontrar_maximo(lista_temps):
    if len(lista_temps) == 0:
        return None
    maximo = lista_temps[0]
    for temp in lista_temps:
        if temp > maximo:
            maximo = temp
    return maximo
```

El orden importa

La validación va **antes** de `lista_temps[0]`: con una lista vacía, esa línea lanzaría `IndexError`.

Nassi-Shneiderman

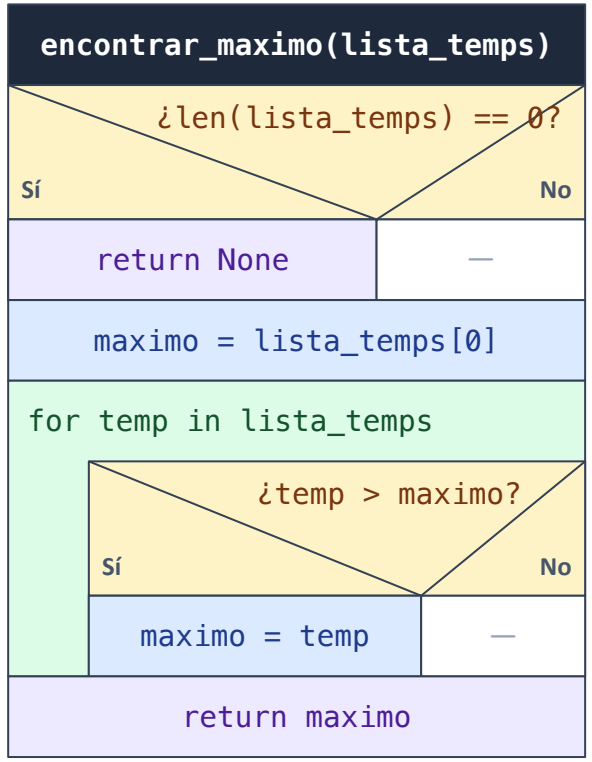
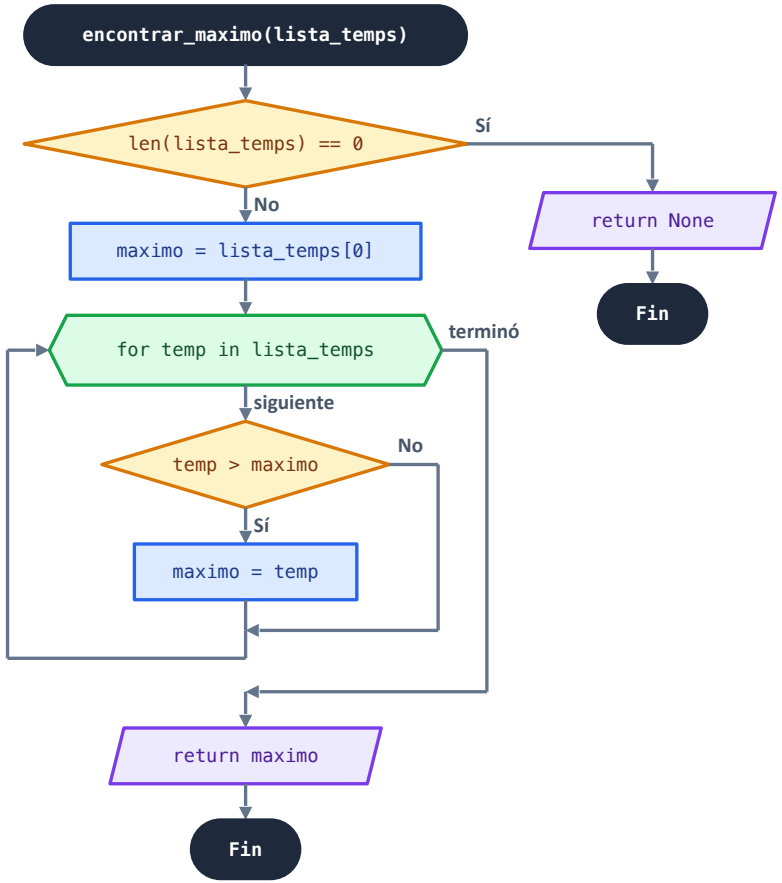


Diagrama de flujo



11 · Promedio

Dos acumuladores en el mismo ciclo y una división al final, fuera del ciclo.

Python · calcular_promedio_goles([2, 0, 4]) → 2.0

```python
def calcular_promedio_goles(lista_goles):
    total = 0
    partidos = 0
    for goles in lista_goles:
        total = total + goles
        partidos = partidos + 1
    return total / partidos
```

Nassi-Shneiderman

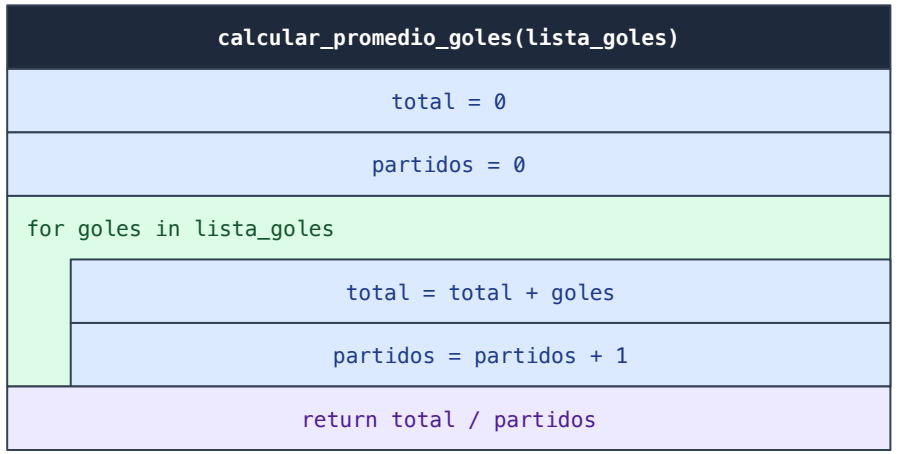


Diagrama de flujo

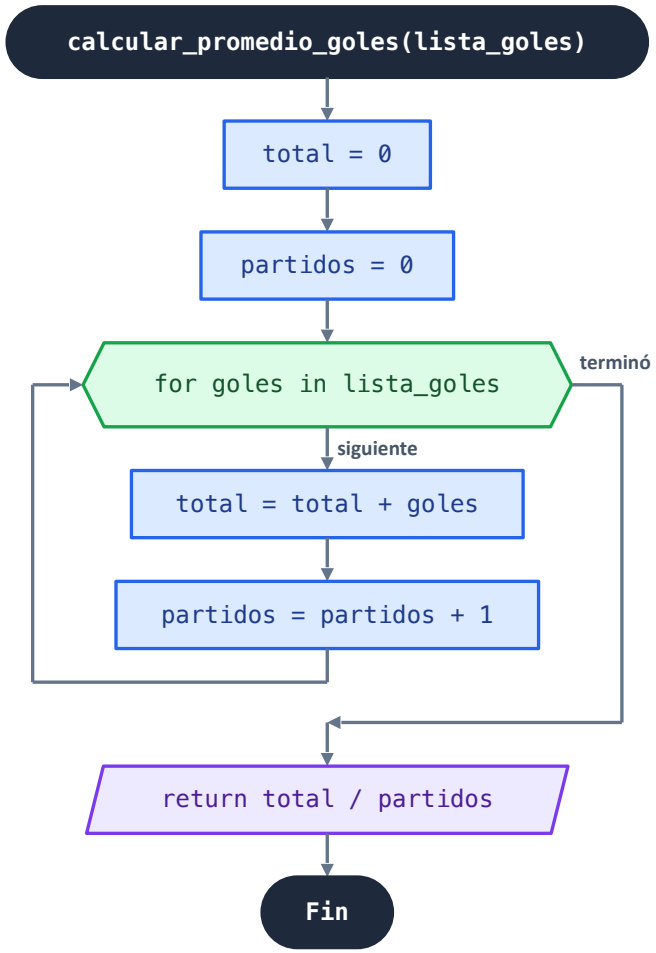


Tabla de equivalencias

Python, diagrama de flujo y Nassi-Shneiderman, renglón por renglón.

Python	Diagrama de flujo	Nassi-Shneiderman
`total = 0`	Rectángulo	Bloque rectangular
`return total`	Paralelogramo y luego Fin	Bloque; la función termina ahí
`if sin else`	Rombo; la rama No rodea el bloque	Triángulo; columna No vacía
`if / else`	Rombo con dos caminos que se juntan	Triángulo con dos columnas
`if / elif / else`	Escalera de rombos	Triángulos anidados en la columna No
`for x in lista:`	Hexágono + flecha de regreso	Bloque en forma de L
`sangría (4 espacios)`	Las cajas entre el rombo y el punto donde se juntan sus ramas	Lo que queda dentro de un bloque

La sangría de Python y el anidado de N-S son la misma idea: lo que está adentro, pertenece.

⚠️ Errores típicos al dibujar

Los que más aparecen cuando se traduce entre código y diagrama.

Rombo con una sola salida

Toda decisión tiene **Sí** y **No**, aunque la rama No no haga nada. Si falta, no se sabe a dónde ir.

Ciclo sin flecha de regreso

Sin la flecha que sube al hexágono, el `for` se ejecutaría una sola vez.

`return` dentro del ciclo

Si el `return` queda dentro del ciclo (antes de la flecha de regreso, o dentro de la L), la función termina en la **primera vuelta**.

Nombre sin verbo

El óvalo de inicio y la barra de N-S también llevan verbo + complemento: `contar_victorias`, no `victorias`.

Práctica

Para cada enunciado, en este orden:



Ejercicio A

Recibe una lista de precios y regresa **cuántos** cuestan más de 100.

Ejercicio B

Recibe una lista de calificaciones y regresa la **más baja**. Si la lista está vacía, regresa `None`.

Ejercicio C

Recibe una temperatura y regresa `"frío"` si es menor a 15, `"templado"` si es menor a 25 y `"calor"` en otro caso.

Ejercicio D

Recibe las horas que dormiste cada día de la semana y regresa el **promedio**.

Referencias

Para quien quiera leer más.

AlSuhaibani, R., Newman, C., Decker, M., Collard, M. y Maletic, J. (2021). On the naming of methods: A survey of professional developers. *ICSE 2021*.

Arnaoudova, V., Di Penta, M. y Antoniol, G. (2016). Linguistic antipatterns: What they are and how developers perceive them. *Empirical Software Engineering*, 21(1), 104–158.

Chen, J.-Y. (2007). Do Chinese and English speakers think about time differently? Failure of replicating Boroditsky (2001). *Cognition*, 104(2), 427–436.

Fausey, C. y Boroditsky, L. (2011). Who dunnit? Cross-linguistic differences in eye-witness memory. *Psychonomic Bulletin & Review*, 18, 150–157.

Feitelson, D. et al. (2022). How developers choose names. *IEEE Transactions on Software Engineering*, 48(1), 37–52.

Goldstine, H. y von Neumann, J. (1947). *Planning and Coding of Problems for an Electronic Computing Instrument*. Institute for Advanced Study.

Hofmeister, J., Siegmund, J. y Holt, D. (2017). Shorter identifier names take longer to comprehend. *SANER 2017*, 217–227.

Høst, E. y Østvold, B. (2009). Debugging method names. *ECOOP 2009*, LNCS 5653, 294–317.

January, D. y Kako, E. (2007). Re-evaluating evidence for linguistic relativity: Reply to Boroditsky (2001). *Cognition*, 104(2), 417–426.

Kernighan, B. y Pike, R. (1999). *The Practice of Programming*. Addison-Wesley.

Lupyan, G. y Ward, E. (2013). Language can boost otherwise unseen objects into visual awareness. *PNAS*, 110(35), 14196–14201.

McConnell, S. (2004). *Code Complete* (2.ª ed.). Microsoft Press.

Nassi, I. y Shneiderman, B. (1973). Flowchart techniques for structured programming. *SIGPLAN Notices*, 8(8), 12–26.

Winawer, J. et al. (2007). Russian blues reveal effects of language on color discrimination. *PNAS*, 104(19), 7780–7785.

Primero el verbo.
Después el dibujo.
Al final, el código.

Desde hoy, toda función: verbo en infinitivo + complemento. Si regresa True / False: es_ + complemento.